



# Cours de Web dynamique : PHP–MySQL

Licence Mathématiques Appliquées

Septembre 2023

# Les bases du langage PHP

Types

Chaînes de caractères

Tableaux

Opérateurs

Instructions conditionnelles

Boucles

Fonctions

Variables

Objets

# Sites de références

## ❖ PHP

- ▶ <http://www.php.net>
  - ▶ <http://www.phpinfo.net>
  - ▶ <http://www.phpfrance.com>
  - ▶ <http://www.developpez.com/php/>
- Pour obtenir en ligne toute la documentation officielle (en français) sur une commande PHP : [http://fr.php.net/nom\\_de\\_la\\_commande](http://fr.php.net/nom_de_la_commande)
  - Exemple : <http://fr.php.net/echo>

## ❖ MySQL

- ▶ <http://www.mysql.com/>
- ▶ <http://dev.mysql.com/doc/refman/5.0/fr/>

# Commentaires et casse

- ❖ On peut mélanger les commentaires du C, C++ et shell !

`<?php /* commentaire type C */`

`// commentaire type C++`

`# commentaire type shell`

`?>`

- ❖ La casse n'intervient pas dans les noms de fonction

`echo "Bonjour";` et

`EcHo "Bonjour";` sont équivalents

- ❖ En revanche, elle intervient dans les noms de variables

`echo $nom;` et

`echo $NOM;` concernent deux variables différentes

# Types de données et variables

- ❖ Les noms de variables sont précédés d'un \$

```
$i = 5;
```

```
echo "$i";
```

- ❖ Pas de déclaration, l'affectation détermine le type de la variable

- ❖ PHP supporte les types de données suivants

- Booléens
- Nombres entiers
- Nombres à virgule flottante
- Chaînes de caractères
- Tableaux
- Objets (programmation orientée objet)

# Booléens, entiers et flottants

## ❖ Booléens

➤ `true` ou `false`

➤ Sont considérés comme faux : le booléen `FALSE`, l'entier `0`, le flottant `0.0`, `""` ou `"0"`, un tableau vide, un objet vide, la constante `NULL`

## ❖ Entiers positifs ou négatifs

`$i = -5; # base 10`

`$i = 012; # base 8`

`$i = 0x12; # base 16`

➤ Pas de division entière mais transtypage (`cast`) possible

`$i = (int)(7/3) # $i vaut 2`

## ❖ Les flottants

`$f = -1.3;`

`$f = 1.2e-3; # 0,0012`

# Chaînes de caractères

## ❖ Chaînes de caractères

```
$ch1 = 'Bonjour'; $ch2 = "$ch1 UVBF!\n";
```

➡ entre guillemets simples, rien n'est interprété sauf \ ' et \ \ qui échappent le caractère ' et le caractère \

➡ entre guillemets doubles, sont interprétés les variables, les caractères spéciaux \n \r \t \\$ \" \\

➡ protection des noms de variables

```
$ch1 = 'Bonjour '; $ch2 = "${ch1}UVBF!\n";
```

➡ `echo "\n";` provoque un saut de ligne dans le code HTML généré mais ne provoque de saut de ligne HTML (il faut utiliser `<br>`) !

# Chaînes de caractères

## ❖ Conversion de chaînes de caractères en valeur numérique

`$i = 1 + "4.5"; # $i vaut 5.5`

`$i = 1 + "-1e3"; # $i vaut -999`

`$i = 1 + "toto + 9"; # $i vaut 1`

`$i = 1 + "9 + toto"; # $i vaut 10`

☞ La chaîne est de type flottant si elle contient '.', 'e' ou 'E' sinon elle est de type entier

☞ La valeur est définie par la première partie de la chaîne (0 si c'est du texte)

## ❖ La constante NULL est convertie en "", un tableau en la chaîne "Array"

`$i = NULL; echo "-$i-"; # affiche "--"`

`$tab[0] = 1; echo "$tab"; # affiche "Array"`



# Chaînes de caractères

## ❖ Accès aux caractères d'une chaîne

```
$ch = 'Ceci est une chaîne.'; echo $ch{3}; # affiche "i"
```

## ❖ Affichage de chaînes

➤ `echo` et `print` pour les affichages simples

➤ `printf` pour les chaînes formatées (comme en C)

➤ `print_r` et `var_dump` pour les variables complexes

```
$nom = "Monsieur";
```

```
echo "Bonjour", " $nom"; # affiche "Bonjour Monsieur"
```

```
print "Bonjour $nom"; # affiche "Bonjour Monsieur"
```

```
printf("Bonjour %s", $nom); # affiche "Bonjour Monsieur"
```

# Chaînes de caractères

## ❖ Substitutions de chaînes

- **addslashes(str)** ajoute un \ devant tous les caractères spéciaux de la chaîne passée en argument - utile pour les requêtes aux bases de données
  - **stripslashes(str)** effectue l'opération inverse (enlève les \)
  - **str\_replace(search, replace, str)** remplace dans **str** toutes les occurrences de **search** par **replace**
- # la commande suivante affiche "Bonjour titi"
- ```
echo str_replace("toto", "titi", "Bonjour toto");
```

# Chaînes de caractères

## ❖ Découpage de chaînes

- ✓ `explode(sep, str)` retourne un tableau de chaînes en scindant `str` à l'aide du séparateur `sep`
- ✓ `implode(sep, tab)` retourne une chaîne fabriquée par la concaténation des éléments du tableau et du séparateur `sep` entre chaque élément (`join()` fait la même chose)
- ✓ `chop(str)` ou `rtrim(str)`, `ttrim(str)`, `ltrim(str)` suppriment des caractères d'espacement dans `str` (" ", \t, \n, \r, \0)
- ✓ `parse_str(str)` analyse une chaîne type QUERY\_STRING et fabrique les couples `$nom=valeur` associés

## ❖ Comparaison et longueur

- ✓ `strcmp(str1, str2)` : comparaison binaire (comme en C)
- ✓ `strlen(str)` retourne la longueur de `str`

# Chaînes de caractères

## ❖ Web

- ♣ `rawurlencode(str)/rawurldecode(str)` encode/décode la chaîne `str` selon la RFC1738 (`%xy`  $\leftrightarrow$  caractère)
- ♣ `urlencode(str)/urldecode(str)` idem mais adapté au formulaire (remplace également le `+` en espace...)
- ♣ `base64_encode(str)/base64_decode(str)` encode/décode la chaîne en type MIME base64 (par exemple pour `user:passwd`)
- ♣ `parse_url(str)` retourne dans un tableau les composants d'une URL (scheme, host, port, user, pass, path, query, fragment)  
`http://username:password@hostname/path?arg=value#anchor`

# Chaînes de caractères

❖ Expressions régulières (substitution et recherche complexes)

♣ PHP gère les expressions régulières POSIX et celles du langage Perl

➡  $\wedge$  : début de ligne

➡  $\$$  : fin de ligne

➡  $\cdot$  : n'importe quel caractère

➡  $[A-Z]$  : caractère appartenant à l'ensemble des majuscules

➡  $[^A-Z]$  : caractère n'appartenant pas à l'ensemble des majuscules

➡  $r^*$  : 0, 1 ou plusieurs occurrences successives de l'expression régulière  $r$

# Chaînes de caractères

❖ Expressions régulières (substitution et recherche complexes)

♣ PHP gère les expressions régulières POSIX et celles du langage Perl

☞  **$r\{m,n\}$**  : entre m et n occurrences successives de l'expression régulière r

☞  **$r\{m\}$**  : exactement m occurrences successives de l'expression régulière r

☞  **$r\{m,\}$**  : au moins m occurrences successives de l'expression régulière r

☞  **$\backslash$**  : caractère d'échappement ( $m\backslash *n = m*n$ )

☞  **$r+$**  : un ou plusieurs exemplaires de r

☞  **$(r1 | r2 | r3)$**  : r1 ou r2 ou r3

# Chaînes de caractères

❖ Expressions régulières (substitution et recherche complexes)

➤ `ereg(profil,str,$tab)` recherche le profil dans `str`

- retourne `true` ou `false`

- remplit `$tab` avec les occurrences trouvées (max 10)

- sensible à la casse

# \$date est au format YYYY-MM-DD

```
if (ereg ("([0-9]{4})-([0-9]{1,2})-([0-9]{1,2})", $date, $regs)) {
```

```
# $regs[0] contient une copie de la chaîne trouvée
```

```
# affichage au format DD/MM/YYYY
```

```
echo "$regs[3]/$regs[2]/$regs[1]"; } else {
```

```
echo "Format de date invalide : $date"; }
```

➤ `eregi(profil,str,$tab)` idem mais insensible à la casse

# Chaînes de caractères

- ❖ Expressions régulières (substitution et recherche complexes)
  - ✓ `ereg_replace(profil, replace, str)` recherche le profil dans `str` et le remplace par `replace`
    - retourne `str` modifiée
    - sensible à la casse
  - ✓ `eregi_replace(profil, replace, str)` idem mais insensible à la casse
  - ✓ `split(profil, str)` retourne un tableau de sous-chaînes de `str` délimitées par le profil  
# MM/DD/YYYY ou MM.DD.YYYY ou MM-DD-YYYY  
`list ($month, $day, $year) = split ('[/.-]', $date);`
  - ✓ `spliti(profil, str)` idem mais insensible à la casse



# Date et heure

## ❖ Afficher la date et l'heure actuelle

```
echo "Nous sommes le " . date("j m Y") . " et il est " . date("H  
 \h i") . " mn";
```

## ❖ Les fonctions `date()` et `mktime()`

- `date("format", $timestamp)` retourne une chaîne de caractères qui contient la date `$timestamp` au format indiqué en premier argument
- `$timestamp` est la date Unix (nombre de secondes depuis le 1er janvier 1970)
- Si `$timestamp` est omis, il s'agit de la date actuelle
- Si le format est omis, `date()` retourne `$timestamp`
- `mktime(h, m, s, M, J, A)` retourne `$timestamp` associé à la date indiquée en paramètres,

# Tableaux

- ❖ Les tableaux de PHP sont associatifs
  - l'index dans le tableau est appelé clé
  - la valeur associée à une clé est appelée valeur
  - un tableau est un ensemble d'associations clé/valeur
  - la clé peut être un entier ou une chaîne de caractères

- ❖ Création d'un tableau

- soit directement en affectant des valeurs au tableau
- soit en utilisant la fonction `array()`

`$tab[0] = 1; # clé entière, valeur entière`

`$tab[1] = "toto"; # clé entière, valeur de type chaîne`

`$tab["toto"] = "titi"; # clé et valeur de type chaîne`

# Tableaux

## ❖ Autres exemples

`$tab["toto"][0] = 1; # tableau à 2 dimensions`

`$tab["toto"][0]["titi"][3] = "toto"; # tout est permis !`

`$tab[] = "tata"; # la clé sera un entier dont la valeur est la  
clé entière la plus grande du tableau + 1`

`$tab1 = array(1, "toto"); # 0=>1 et 1=>"toto"`

`$tab2 = array("toto"=>1, "titi"=>"toto"); # la clé est donnée`

## ❖ Nombre d'éléments d'un tableau

`sizeof($tab); count($tab); # retourne le nombre d'éléments du  
tableau $tab s'il existe, 1 si $tab n'est pas un tableau, 0 si $tab n'existe  
pas`

# Tableaux

## ❖ Suppression d'un élément

`unset($tab["toto"]);` # fonctionne aussi pour une variable

## ❖ Tris de tableaux

- le tri peut se faire sur les clés et/ou les valeurs ; l'association clé/valeur peut être cassée
- `asort()/arsort()` : trie le tableau par ordre croissant/décroissant de valeurs
- `ksort()/krsort()` : trie le tableau par ordre croissant/décroissant de clés
- `sort()` : trie le tableau par ordre croissant de valeurs et réassigne des clés (0,1,...) ; on perd l'association clé/valeur
- `uasort()/uksort()/usort()` : identiques à leur homologue mais la fonction de comparaison est fournie

# Tableaux

## ❖ Le pointeur de tableau

- ✓ à chaque tableau correspond un pointeur interne qui est une référence sur l'élément courant
- ✓ `current($tab)` désigne l'élément courant
- ✓ `next($tab)` déplace le pointeur vers l'élément suivant
- ✓ `prev($tab)` déplace le pointeur vers l'élément précédent
- ✓ `end($tab)` déplace le pointeur sur le dernier élément
- ✓ `reset($tab)` déplace le pointeur sur le premier élément

```
$tab = array("a"=>1, "d"=>5, "b"=>8, "c"=>4);  
$val = current($tab); echo "$val<br>"; # affiche "1<br>"  
$val = next($tab); echo "$val<br>"; # affiche "5<br>"  
asort($tab); end($tab); prev($tab); $val = prev($tab);  
echo "$val<br>"; # affiche "4<br>"
```

# Tableaux

## ❖ Extraction d'éléments d'un tableau

- `list()` permet d'extraire des valeurs d'un tableau

```
$tab = array(1, 8, 5); sort($tab);  
list($v1, $v2) = $tab;  
echo "$v1 $v2"; # affiche "1 5"
```

- `key($tab)` permet d'extraire la clé de l'élément pointé par le pointeur interne du tableau

```
$tab = array("a"=>1,"b"=>8); next($tab);  
$clé = key($tab); $val = $tab[$clé];  
echo "$clé: $val"; # affiche "b: 8"
```

- `extract($tab)` permet d'extraire d'un tableau toutes les valeurs, chaque valeur est copiée dans une variable ayant pour nom la valeur de la clé

```
$tab = array("a"=>1,"b"=>8); extract($tab);  
echo "$b $a"; # affiche "8 1"
```

# Tableaux

## ❖ Extraction d'éléments d'un tableau

✓ `each($tab)` retourne la paire clé/valeur courante du tableau et avance le pointeur de tableau ; cette paire est retournée dans un tableau de 4 éléments : 0=>clé, 1=>valeur, key=>clé et value=>valeur

## ❖ Parcours de l'ensemble du tableau

✓ avec `list()` et `each()`

```
$tab = array("a"=>1, "d"=>5, "b"=>8, "c"=>4);
```

```
reset ($tab);
```

```
while (list ($k, $v) = each ($tab)) {
```

```
echo "$k => $v<br />\n"; }
```

#quand le pointeur dépasse la fin de \$tab, each retourne false

# Tableaux

## ❖ Parcours de l'ensemble du tableau

- `foreach()` place le pointeur en tête du tableau et parcourt l'ensemble des éléments ; `foreach()` travaille sur une **copie** du tableau original

```
$tab = array("a"=>1, "d"=>5, "b"=>8, "c"=>4);
```

```
# parcours des valeurs uniquement : $v = 1, 5, 8, 4
```

```
foreach ($tab as $v) { echo "valeur : $v<br>\n"; }
```

```
# parcours des couples clé/valeur
```

```
foreach ($tab as $k => $v) { echo "$k : $v<br>\n"; }
```

```
# tableaux multidimensionnels
```

```
$tab2[0][0] = "a"; $tab2[0][1] = "b";
```

```
$tab2[1][0] = "y"; $tab2[1][1] = "z";
```

```
foreach($tab2 as $t) { foreach ($t as $v) { echo "$v\n"; } }
```



# Les constantes

- ❖ Constantes définies par le programmeur

```
define ("MAX", 255); echo MAX;
```

```
define ("NOM", "Durand"); echo NOM;
```

- ❖ Principales constantes définies par PHP

- ☐ `__FILE__` : chemin absolu du fichier en cours d'exécution

- ☐ `__LINE__` : numéro de la ligne en cours d'exécution

- ☐ `PHP_VERSION` : version de PHP qui est utilisée

- ☐ `PHP_OS` : OS de la machine (ex : Linux)

- ☐ `TRUE` et `FALSE`

- ☐ `E_*` : gestion des erreurs (`E_ALL` = toutes les erreurs)

- ☐ Il y en a plein d'autres !

# Les erreurs

## ❖ 4 types d'erreurs/alertes en PHP

- ✓ `E_ERROR` (1) : erreur d'exécution
- ✓ `E_WARNING` (2) : alerte
- ✓ `E_PARSE` (4) : erreur d'analyse
- ✓ `E_NOTICE` (8) : notes (alertes pouvant être ignorées)

## ❖ Changement du niveau d'erreurs

- ✓ `error_reporting(cst)` permet de changer le niveau d'erreurs reportées (par défaut  $7=1+2+4$ ) dans le script
- ✓ directive `error_reporting` dans *php.ini* ou *httpd.conf*

## ❖ `error_log()` permet d'envoyer un message d'erreur dans les logs du serveur web, à un port TCP, dans un fichier, par mail, ...

# La valeur NULL

- ❖ Constante spéciale qui représente l'absence de valeur
- ❖ Valeur insensible à la casse, introduite en PHP4

```
$var = NULL;
```

```
isset($var) # retourne FALSE
```

```
is_null($var) # retourne TRUE
```

```
empty($var) # retourne TRUE
```

```
$ch = "";
```

```
isset($ch) # retourne TRUE
```

```
is_null($ch) # retourne FALSE
```

```
empty($ch) # retourne TRUE
```

# Les opérateurs

## ❖ Opérateurs arithmétiques

- ❑ addition :  $\$a + \$b$
- ❑ soustraction :  $\$a - \$b$
- ❑ multiplication :  $\$a * \$b$
- ❑ division :  $\$a / \$b$
- ❑ modulo (reste de la division entière) :  $\$a \% \$b$

```
 $\$i = 7/3$ ; echo "<br>$i"; # affiche 2.333333333333333
```

```
 $\$j = 7\%3$ ; echo "<br>$j"; # affiche 1
```

## ❖ Concaténation de chaîne de caractère (.)

```
 $\$ch1 = "Bonjour "$ ;  $\$ch2 = "Monsieur"$ ;
```

```
 $\$ch = \$ch1.\$ch2$ 
```

```
echo  $\$ch . " Durand" . ' !'$ ;
```

```
# affiche "Bonjour Monsieur Durand !"
```

# Les opérateurs

## ❖ Opérateurs binaires

- ❑ ET bit à bit :  $a \& b$
- ❑ OR bit à bit :  $a | b$
- ❑ XOR bit à bit :  $a \wedge b$
- ❑ NON bit à bit :  $\sim a$
- ❑ décalage à droite de  $b$  bits :  $a >> b$
- ❑ décalage à gauche de  $b$  bits :  $a << b$

## ❖ Opérateurs logiques

- ❑ ET logique : **and**, **&&** (les deux sont possibles)
- ❑ OU logique : **or**, **||**
- ❑ XOR logique : **xor**
- ❑ NON logique : **!**

# Les opérateurs

## ❖ Opérateurs d'affectation

❑ affectation avec le signe =

`$a = ($b = 4) + 1; # $b vaut 4 et $a vaut 5`

❑ les opérateurs combinés : `+=`, `-=`, `*=`, `/=`, `.=`, `&=`, `|=`,  
`^=`, `<<=`, `>>=`, `~=`

`$a += 1; # équivalent à $a = $a + 1;`

`$ch .= "!"; # équivalent à $ch = $ch . "!";`

❑ `++` est équivalent à `+= 1`, `--` est équivalent à `-= 1`

`$i = 0;`

`echo ++$i; # incrémente $i puis affiche 1`

`echo $i++; # affiche 1 puis incrémente $i`

`echo $i; # affiche 2`

# Les opérateurs

## ❖ Opérateurs de comparaison

- Égal à :  $a == b$
- Différent de :  $a != b$
- Inférieur à :  $a < b$
- Supérieur à :  $a > b$
- Inférieur ou égal à :  $a \leq b$
- Supérieur ou égal à :  $a \geq b$

## ❖ Opérateur ternaire

- $(condition) ? (expr1) : (expr2);$
  - renvoie  $expr1$  si condition est vraie sinon renvoie  $expr2$
- $max = (a \geq b) ? a : b; \# \text{ } max \leftarrow \max(a, b)$

# Les instructions conditionnelles

❖ if/then/else comme en C

```
if (condition1) {  
    # si condition1 est vraie  
}  
elseif (condition2) {  
    # si condition2 est vraie (et pas condition1)  
}  
elseif (condition3) {  
    # si condition3 est vraie (et ni condition1, ni condition2)  
}  
else {  
    # si ni condition1, ni condition2, ni condition3 ne sont  
    vraies  
}
```



# Les instructions conditionnelles

❖ if/then/else avec prototype simplifié

if (condition1) :

# si condition1 est vraie

elseif (condition2) :

# si condition2 est vraie (et pas condition1)

elseif (condition3) :

# si condition3 est vraie (et ni condition1, ni condition2)

else :

# si ni condition1, ni condition2, ni condition3 ne sont vraies

endif;

# Les instructions conditionnelles

❖ Le switch comme en C

```
switch(expr) {  
  case (val1) :  
    # à exécuter si expr vaut val1  
  break;  
  case (val1) :  
    # à exécuter si expr vaut val2  
  break;  
  default :  
    # à exécuter dans tous les autres cas  
}
```

# Les structures répétitives (boucles)

## ❖ Boucle "tant que"

➤ comme en C

```
while (condition) {  
    # exécuté tant que la condition est vraie  
}
```

➤ prototype simplifié

```
while (condition) :  
    # exécuté tant que la condition est vraie  
endwhile;
```

## ❖ Boucle "do...while" comme en C

```
do { /*...*/ } while (condition);
```

# Les structures répétitives (boucles)

## □ Boucle "for"

- comme en C

```
for ($i = 1; $i <= 10; $i++) {  
    # exécuté tant que ($i <= 10) est vraie  
}
```

- prototype simplifié

```
for ($i = 1; $i <= 10; $i++) :  
    # exécuté tant que ($i <= 10) est vraie  
endfor;
```

□ L'instruction **break** permet de sortir d'une boucle

□ L'instruction **continue** permet de passer directement à l'itération suivante de la boucle

# Les fonctions

- ❖ En PHP3, une fonction devrait être définie avant d'être utilisée ; ce n'est pas le cas en PHP4

```
function max($a, $b) {  
    # corps de max()  
    if ($a>$b) { return $a; } else { return $b; }  
}  
  
# appel de la fonction max()  
$m = max(5, 8); # $m vaut 8
```

- ❖ Il est possible de retourner n'importe quel type de variable (tableau, objet, ...)
- ❖ Une fonction peut ne rien retourner

# Les fonctions

```
<script language="php">
```

```
function ecritTexteItalique($var)
```

```
{ print("<i> $var </i>");
```

```
}
```

```
ecritTexteItalique("Ceci est écrit en italique");
```

```
</script>
```

ou :

```
<script language="php">
```

```
function texteItalique($var)
```

```
{ $v = "<i>";
```

```
$v .= $var;
```

```
$v .= "</i>";
```

```
return($v);
```

```
}
```

```
print(texteItalique("Ceci est écrit en italique"));
```

```
</script>
```

# Les fonctions

- ❖ Les arguments de fonction - PHP supporte :
  - ✓ Le passage d'arguments par valeur (par défaut)
  - ✓ Le passage d'arguments par référence
  - ✓ Les valeurs par défaut des arguments
  - ✓ Un nombre variable d'arguments (PHP4 uniquement)
- ❖ Deux types de passage par référence
  - ✓ De façon permanente en ajoutant un & devant le nom de la variable dans la définition de la fonction
  - ✓ De façon ponctuelle en ajoutant un & devant le nom de la variable lors de l'appel de la fonction

# Les fonctions

```
function plus3($a) {
```

```
    $a += 3;
```

```
}
```

```
function plus5(&$a) {
```

```
    $a += 5;
```

```
}
```

# passage par valeur

```
$x = 1; plus3($x); echo $x; # affiche "1"
```

# passage par référence ponctuel

```
$y = 1; plus3(&$y); echo $y; # affiche "4"
```

# passage par référence "forcé"

```
$z = 1; plus5($z); echo $z; # affiche "6"
```



# Les fonctions

- ❖ PHP permet de définir comme en C++ des valeurs par défaut pour les arguments
  - La valeur par défaut d'un argument doit être une constante (ni une variable, ni un membre de classe)
  - Les arguments avec valeur par défaut doivent être placés après les arguments sans valeur par défaut

```
function concat($sch1, $sch2 = "beau") {  
    return $sch1.$sch2;  
}
```

```
echo concat("Il fait "); # affiche "Il fait beau"
```

```
echo concat("Il fait ", "froid"); # affiche "Il fait froid"
```

# Les fonctions

- ❖ PHP4 supporte les fonctions à nombre d'arguments variable
  - ❑ `func_num_args()` retourne le nombre d'arguments passé à la fonction
  - ❑ `func_get_arg(arg_num)` retourne l'argument à la position `arg_num` dans la liste des paramètres (à partir de 0) ; retourne `false` si `arg_num` est trop grand
  - ❑ `func_get_args()` retourne un tableau qui contient la liste des arguments

```
function foo() {  
    $numargs=func_num_args(); $arg_list=func_get_args();  
    for ($i = 0; $i < $numargs; $i++) {  
        echo "L'argument $i est: " . $arg_list[$i] . "<br>\n";  
    }  
}  
foo(1, 2, 3); foo("toto");
```

# Les fonctions

- ❖ PHP4 permet le retour de fonction par référence

```
function &retourne_reference() {  
    return $une_ref;  
} # $new_ref est une référence sur $une_ref  
$new_ref = &retourne_reference();
```

- ❖ Les variables fonctions

```
function call_back($arg = 'dans call_back') {  
    echo "$arg";  
}  
$func = 'call_back';  
$func(); # affiche "dans call_back"  
$func("func est call_back"); # affiche "func est call_back"
```

# Les fonctions récursives

Exemple : calcul de  $n!$

<?

```
function factorielle($n)  
{ if ($n > 1)  
    return($n * factorielle($n-1));  
else  
    return(1);  
  
}
```

?>

# Les variables

- ❖ Les noms de variables ne doivent pas commencer par un chiffre
- ❖ PHP4 permet des affectations par référence

```
$x = 2; $y = &$x; $y++; echo $x; #affiche 3
```

- ❖ Variables prédéfinies

- ☐ depuis PHP 4.1.0, des tableaux "super-globaux" sont prédéfinis

- ✓ accessibles dans n'importe quel contexte d'exécution
  - ✓ infos sur les variables du serveur, les variables d'environnement, les variables de PHP, ...

- ☐ le contenu de ces tableaux dépend de la version et de la configuration du serveur et de PHP

# Les variables

## ❖ Variables prédéfinies - tableaux globaux PHP4.1.0

- **\$GLOBALS** : variables globales de l'exécution en cours
- **\$\_SERVER** : variables fournies par le serveur WEB
  - **'PHP\_SELF'** (chemin du script sur le serveur),
- **\$\_GET** : variables fournies par HTTP/méthode GET
- **\$\_POST** : variables fournies par HTTP/méthode POST
- **\$\_COOKIE** : variables issues des cookies HTTP reçus
- **\$\_FILES** : variables fournies par HTTP suite à un téléchargement de fichier(s) par la méthode POST

# Les variables

## ❖ Variables prédéfinies - tableaux globaux PHP4.1.0

- `$_ENV` : variables d'environnement positionnées au démarrage du serveur Web
- `$_REQUEST` : tableau associatif constitué de toutes les variables en entrée (contient en particulier `$_GET`, `$_POST`, `$_COOKIE`, `$_FILES`)
- `$_SESSION` : variables relatives à la session en cours
- avant PHP4.1.0 (toujours disponibles mais obsolète) :
  - ✓ `$HTTP_SERVER_VARS`, `$HTTP_GET_VARS`, `$HTTP_POST_VARS`, `$HTTP_COOKIE_VARS`, `$HTTP_POST_FILES`, `$HTTP_ENV_VARS`, `$HTTP_SESSION_VARS`
  - ✓ pas super-globaux -> `global $HTTP_GET_VARS`
- Si `register_globals = on` dans `php.ini`, les variables prédéfinies sont directement accessibles par leur nom

(ex: `echo $PHP_SELF;`)

# Les variables

❖ `import_request_variables(type, prefix)` permet d'importer les variables de `_GET`, `_POST`, ou `_COOKIE` en tant que variables globales (type = 'G', 'P' ou 'C') : utile si `register_globals = off`

❖ Variables globales

```
<?php $a = 1; include "file1.inc"; ?>
```

➤ \$a est globale ; elle est accessible dans file1.inc

❖ Variables locales à une fonction

```
<?php
$a = 1; # portée globale
function affiche() {
    echo $a; # $a est locale à la fonction
}
affiche(); # n'affiche rien !
?>
```



# Les variables

## ❖ Accès à une variable globale dans un bloc

- ❑ Déclarer la variable comme **global** dans le bloc (crée une référence locale sur la variable globale)
- ❑ Utiliser le tableau associatif **\$GLOBALS**

```
<?php $a1 = 1; $a2 = 2; # portée globale  
function affiche() {  
    global $a1; # $a1 est la variable globale  
    echo $a1." et ".$GLOBALS['a2'];  
    $a1++;  
}  
affiche(); # affiche "1 et 2"  
echo $a1; # affiche "2" ($a1 a été modifiée)  
?>
```

# Les variables

## ❖ Variables statiques dans un bloc

- Une variable statique a une portée locale à un bloc mais conserve sa valeur entre deux exécutions du bloc
- Elles sont utiles pour les fonctions récursives

```
function up_to_10() {  
    static $cpt = 0;  
    $cpt++;  
    echo "$cpt ";  
    if ($cpt < 10) up_to_10();  
}  
up_to_10(); # affiche 1 2 3 4 5 6 7 8 9 10
```

# Les variables

## ❖ Variables dynamiques

- ❑ une variable dynamique prend la valeur d'une variable et l'utilise comme nom d'une autre variable

```
$nom_var = "x"; $$nom_var = 5;
```

```
echo "$nom_var $x"; # affiche "x 5"
```

```
echo "$nom_var ${$nom_var}"; # affiche aussi "x 5"
```

- ❑ ambiguïté avec les tableaux : `$$a[1]` peut représenter `${$a}[1]` ou bien `${$a[1]}` (il faut utiliser les accolades)

# Les variables

## ❖ Affichage des variables complexes

- `var_dump()` affiche les informations structurées d'une variable (type, valeur, ...)
- `print_r()` affiche les informations lisibles pour une variable (par exemple affiche les clés et les valeurs d'un tableau)
- attention : `print_r` place le pointeur de tableau à la fin

```
$a = array (1, 2, "a"); var_dump ($a);
```

```
/* affichage :
```

```
array(3) {  
    [0]=> int(1)  
    [1]=> int(2)  
    [2]=> string(1) "a"  
} */
```

# Les variables

## ❖ Variables et types

- ✓ `is_array()`, `is_bool()`, `is_callable()`, `is_double()`,  
`is_float()`, `is_int()`, `is_integer()`, `is_long()`, `is_null()`,  
`is_numeric()`, `is_object()`, `is_real()`, `is_resource()`,  
`is_scalar()`, `is_string()`, `gettype()`, `settype()`
- ✓ `gettype($var)` retourne le type de `$var` : **integer**, **double**,  
**string**, **array**, **object**, ou unknown type

```
$foo = "5bar"; // chaîne
```

```
$bar = true; // booléen
```

```
settype($foo, "integer"); // $foo vaut maintenant 5
```

```
settype($bar, "string"); // $bar vaut maintenant "1"
```

# Les transtypages

❖ Comme en C - les transtypages possibles sont

❑ (int) ou (integer) pour le type entier

❑ (bool) ou (boolean) pour le type booléen

❑ (real), (double) ou (float) pour le type réel

❑ (string) pour le type chaîne

❑ (array) pour le type tableau

❑ (object) pour le type objet

❖ Attention : le transtypage n'a pas toujours une valeur prévisible

# Les objets

## ❖ Définition d'une classe

```
class Caddie {  
    # attributs  
    var $elements;  
    var $owner;  
    # constructeur  
    function Caddie() {  
        $this->owner = $_POST['prenom']. $_POST['nom'];  
        $this->ajoute(1, "cadeau");  
    }  
    # méthode qui ajoute $nb articles de type $type  
    function Ajoute($nb, $type) {  
        $this->elements[$type] += $nb;  
    }  
}
```

# Les objets

❖ Création d'un objet de la classe  
`$client = new Caddie; # appel le constructeur`

❖ Accès aux attributs et méthodes  
`echo $client->owner;`  
`$client->Ajoute(2, "cadeau");`

❖ Héritage de classe  
`class Caddie_prix extends Caddie {`  
    `var $prix; var $les_prix;`  
    `function set_price($type, $value) {`  
        `$this->les_prix[$type] = $value;`  
    `}`  
    `function calcul_prix() {`  
        `$this->prix = 0;`  
        `foreach ($this->elements as $t => $n) {`  
            `$this->prix += $n*$this->les_prix[$t]; }`  
        `}`  
    `}`  
`}`



# Les objets

## ❖ Particularités de PHP :

- Tous les attributs et méthodes sont publics
- Les objets sont libérés automatiquement quand ils ne sont plus utilisés
- Il n'y a pas de destructeur